Predictive TCP

A lightweight machine-learning congestion predictor for TCP, explained from zero

What this document is. A complete, beginner-friendly guide to the project: the networking ideas behind it, what was built, how every part works, how it was built step by step, what the results are, what its weaknesses are, and how to run it yourself. No prior knowledge of networks or machine learning is assumed.

The question	Can a TCP sender see a network traffic jam coming and slow down before packets are lost?
The answer built	Predictive TCP: a new TCP version that runs a tiny decision tree (4 rules) every 100 ms to predict congestion 0.5 s ahead.
The headline result	About 90% fewer lost packets and 26-37% less delay than TCP Reno, while keeping 98-100% of its speed.
The main weakness	When it shares a link with aggressive CUBIC senders it gets only 36% of the bandwidth (a fair share is 50%).

Authors: Antra (24BDS0066) · Anuska Sen (24BCE2219) · Varshini Harikrishna (24BCE0780)
Course: BCSE308L Computer Networks, VIT
Repository: github.com/Antra1705/CN
Tools: NS-3.48 network simulator (C++), Python 3.13, scikit-learn, pandas, NumPy, Matplotlib, Streamlit

Contents

1. The Whole Project in One Page 4

2. Networking Basics You Need 5

2.1 Packets, links and routers 5

2.2 The bottleneck and its queue 5

2.3 What TCP does 5

2.4 Round-trip time (RTT), and why it reveals the queue 5

2.5 How classic TCP behaves: the saw-tooth 6

2.6 The five existing TCP versions compared in this project 6

2.7 Bandwidth-delay product (BDP) 6

3. The Idea Behind Predictive TCP 7

3.1 Why predict 0.5 seconds ahead? 7

3.2 Why use machine learning at all? 7

3.3 Why the model must be tiny 7

3.4 One important rule: only use what the sender can measure 7

3.5 The overall plan 7

4. Tools and Why They Were Used 8

4.1 Why a simulator instead of real computers? 8

5. The Simulated Network 9

5.1 The dumbbell topology 9

5.2 The three congestion levels 9

5.3 Settings shared by every TCP version 9

5.4 Seeds: how the experiments are repeated 9

5.5 What is measured 10

6. The Machine-Learning Part 11

6.1 What kind of machine learning? 11

6.2 Building the dataset 11

6.3 The features (the model's inputs) 11

6.4 What a decision tree is 12

6.5 The tree that was learned 13

6.6 Training and test data 13

6.7 How good is the model? 13

6.8 Choosing the model inside the simulator 14

6.9 Turning the tree into C++ . . . 15

7. How Predictive TCP Works . . . 16

7.1 Step by step . . . 16

7.2 Safety rules . . . 16

7.3 The competition detector . . . 17

7.4 Settings you can change . . . 17

8. How the Project Was Built, Phase by Phase . . . 18

8.1 Problems found along the way, and how they were solved . . . 18

8.2 Rules followed throughout . . . 19

9. Results . . . 20

9.1 Headline: Predictive TCP compared with Reno . . . 20

9.2 All six versions side by side . . . 20

9.3 Against the delay-based and model-based versions . . . 21

9.4 Is it overfitted? The unseen seeds . . . 22

9.5 Success criteria . . . 23

9.6 Which part matters? (ablation) . . . 23

9.7 Different queue sizes . . . 23

9.8 Sharing the link with CUBIC . . . 23

10. Honest Limitations . . . 25

11. How to Run It Yourself . . . 26

11.1 What is in the project folder . . . 26

11.2 One-time setup (macOS) . . . 26

11.3 Run everything . . . 27

11.4 Identical results on any computer (reference mode) . . . 27

11.5 Run a single simulation . . . 27

11.6 The interactive app . . . 28

12. Quick Questions and Answers . . . 29

13. Glossary . . . 30

References . . . 31

1. The Whole Project in One Page

The internet moves data in small pieces called **packets**. When many computers send packets through the same link, the router in front of that link has to make packets wait in a line, called a **queue**. If the line gets too long, the router has no room left and simply throws new packets away (a **drop**). Lost packets must be sent again, and every packet that waits in a long line arrives late.

TCP is the set of rules computers use to send data reliably. It also decides how fast to send. The classic versions of TCP (Tahoe, Reno, CUBIC) only slow down *after* a packet has been lost. By then the queue has already overflowed. So they keep filling the queue until it overflows, slow down, speed up again, and repeat. The result is repeated packet loss and long delays.

This project asks a simple question: **can the sender notice the queue filling up before it overflows, and back off early?** It can, because as the queue grows, every packet takes longer to make the round trip. The project does four things:

1. **Measures** five well-known TCP versions (Tahoe, Reno, CUBIC, Vegas, BBR) in **NS-3**, a network simulator, under low, medium and high traffic.
2. **Trains a small machine-learning model**, a decision tree with only 4 rules, that looks at the sender's own timing measurements and predicts whether the network will be congested **0.5 seconds from now**.
3. **Builds a new TCP version, Predictive TCP**, that runs this tree inside the sender every 100 ms and gently reduces its sending rate when it predicts congestion.
4. **Tests it honestly**: against all five TCP versions, on simulation runs the model never saw, with different router queue sizes, and while sharing the link with CUBIC.

> **Result in one sentence**
>
> Compared with TCP Reno, Predictive TCP loses about **90% fewer packets** and has **26-37% lower delay**, while keeping **98-100% of Reno's speed**. Its main weakness is that it gives up bandwidth when it shares a link with aggressive CUBIC senders.

> **Which machine-learning model was used?**
>
> A single **Decision Tree** (scikit-learn's `DecisionTreeClassifier`) with depth 2 and 4 leaves. It is **not** a Random Forest. A single small tree was chosen because it can be turned into a few lines of C++ `if/else` code that runs inside the simulated TCP sender. It was compared against **Logistic Regression** and a simple **loss-counting rule**, and it beat both. Section 6 explains everything about it.

2. Networking Basics You Need

This section explains every networking idea the rest of the document uses. If you already know how TCP congestion control works, you can skip to Section 3.

2.1 Packets, links and routers

Data such as a video or a web page is cut into packets. In this project each packet carries **1448 bytes** of data. Packets travel over **links** (cables or wireless connections). Each link has a **bandwidth**, how many bits per second it can carry, for example 10 Mbps (10 million bits per second), and a **delay**, how long a bit takes to travel along it. **Routers** sit where links meet and forward each packet towards its destination.

2.2 The bottleneck and its queue

When fast links feed into a slower one, the slower link limits everyone. It is called the **bottleneck**. Packets that arrive faster than the bottleneck can send them wait in the router's **queue** (also called a **buffer**). In this project the queue holds **50 packets** and is **FIFO** (first in, first out). When all 50 places are taken, any new packet is dropped.

> **An everyday picture**
> Think of a toll booth on a highway. Cars (packets) arrive from several roads. The booth (bottleneck) can only let a fixed number of cars through per minute. Extra cars wait in a line (queue). The line has space for only 50 cars; any car arriving when the line is full is turned away (dropped) and has to come back later (be retransmitted). A long line also means every car waits longer (delay).

2.3 What TCP does

TCP (Transmission Control Protocol) delivers data reliably. The receiver sends back a small **acknowledgement (ACK)** for data it has received. If the sender sees that a packet never arrived, it sends it again; this is a **retransmission**.

TCP also decides **how fast to send**. It keeps a number called the **congestion window (cwnd)**: how many packets it may have "in flight" (sent but not yet acknowledged) at once. A bigger window means faster sending. Roughly:

> **sending rate ≈ congestion window ÷ round-trip time**

Changing this window up and down to match what the network can carry is called **congestion control**. Every TCP version in this project differs only in *how* it changes the window.

2.4 Round-trip time (RTT), and why it reveals the queue

The **round-trip time (RTT)** is the time from sending a packet to receiving its ACK. When the queue is empty, RTT equals just the travel time along the links; this is the **base RTT**. In this project's network the base RTT is **44 ms**.

Every packet waiting in the queue adds waiting time. At 10 Mbps, sending one 1448-byte packet takes 1448 × 8 bits ÷ 10,000,000 bits/s ≈ **1.16 ms**. So each packet in the queue adds about 1.16 ms to the

RTT, and a full queue of 50 packets adds about 58 ms, giving an RTT near 102 ms. That is exactly what Reno shows under high load (101 ms).

> **The key insight of the whole project**
>
> **RTT rises before any packet is lost.** As the queue fills from 0 towards 50 packets, the RTT climbs from 44 ms towards 102 ms. A sender that watches its RTT can see the queue growing and react *before* the queue overflows.

2.5 How classic TCP behaves: the saw-tooth

Classic TCP works in a repeating cycle:

- **Slow start:** at the beginning the window doubles every RTT, to find the available speed quickly.
- **Congestion avoidance:** after that the window grows by about 1 packet every RTT.
- **Loss:** the window keeps growing until the queue overflows and a packet is dropped. The sender notices the loss and cuts its window (Reno halves it).
- **Repeat:** the window grows again and the queue fills again.

Drawn over time, the window looks like the teeth of a saw. The problem is that this design *needs* losses to work: loss is its only signal that the network is full. The queue is pushed to the top again and again, so there are repeated drops and the RTT stays high.

2.6 The five existing TCP versions compared in this project

Version	Year / origin	How it reacts to congestion
Tahoe	1988, Jacobson	On a loss, the window falls to 1 packet, then slow start again. Very cautious.
Reno	1990	On a loss, the window is **halved** ("fast recovery"). The standard reference point in this project.
CUBIC	2008, Linux default	Grows the window along a cubic curve, which fills fast links quickly; on a loss it cuts to 70%. Aggressive.
Vegas	1995, Brakmo and Peterson	**Delay-based:** compares expected and actual speed and tries to keep only a few packets queued. Few losses, but tends to lose out to aggressive senders.
BBR (v1)	2016, Google	**Model-based:** estimates the link bandwidth and the base RTT and sends at that rate. Does not slow down on loss, so it can overflow small queues.

NS-3 includes CUBIC, Vegas and BBR, but **not Tahoe or classic Reno**, so the team wrote those two from scratch in C++ (Section 8).

2.7 Bandwidth-delay product (BDP)

The **bandwidth-delay product** is how many packets fit "on the wire" during one round trip: 10 Mbps × 44 ms = 55,000 bytes ≈ **38 packets**. A sender with a window of 38 packets keeps the link exactly full with an empty queue. Any window above that only adds packets to the queue. The project's 50-packet queue is about 1.3 × the BDP.

3. The Idea Behind Predictive TCP

A careful driver does not wait for a crash before braking. They watch the brake lights ahead and slow down early. Predictive TCP does the same thing: it watches its own RTT, which is its "brake lights", and slows down a little when a jam is coming.

3.1 Why predict 0.5 seconds ahead?

Reacting to the queue *right now* is often too late, because packets already in flight still arrive at the router. Predicting **0.5 s ahead** (about 5-10 round trips) gives the sender enough time to slow down before the overflow. The project also tried 0.2 s and 1.0 s (Section 6.8); 0.5 s worked best inside the simulator.

3.2 Why use machine learning at all?

One could hand-pick a rule such as "slow down when RTT is 30% above the base". But which threshold? And what about a queue that is big but already shrinking? Machine learning **learns these thresholds from data**: the team recorded over 100,000 moments from simulations where the true queue size was known, and let the algorithm find the rule that best predicts congestion half a second later.

3.3 Why the model must be tiny

The model runs **inside the TCP sender, every 100 ms, for every connection**. It must be fast, simple and written in the same language as the simulator (C++). A decision tree with 4 leaves becomes a handful of `if/else` lines and costs almost nothing to run. It is also easy to read and explain, so you can see exactly what it learned.

3.4 One important rule: only use what the sender can measure

A real sender cannot look inside the router. So the model's inputs are **only measurements the sender makes itself** (its RTT and how the RTT changes). The router's real queue size is used for one thing only: to **label** the training data with the correct answer.

3.5 The overall plan

The figure below shows the whole pipeline, from simulations to the final report. Each box is one stage; the rest of this document walks through them in order.

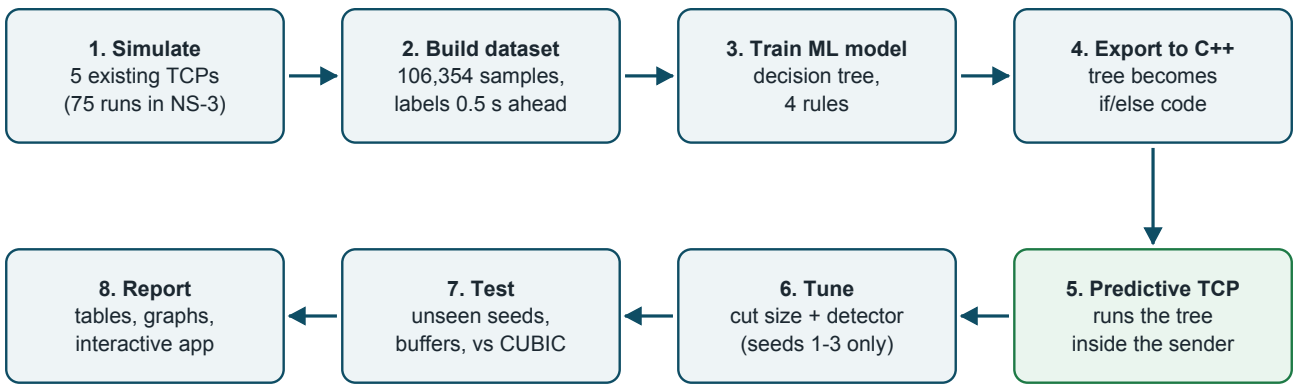


Figure 1. The project pipeline. Stages 1-4 create the model; stages 5-8 put it to work and test it.

4. Tools and Why They Were Used

Tool	What it is	What it does in this project
NS-3.48	A free, widely used network simulator, written in C++	Simulates the routers, links, queue and every TCP sender, packet by packet. All TCP versions, including Predictive TCP, run inside it.
C++	The programming language NS-3 is written in	The network setup, the Tahoe/Reno classes and Predictive TCP are written in C++ (folder `ns3/`).
Python 3.13	A general programming language	All analysis: building the dataset, training, tuning, comparing results, drawing graphs (folder `analysis/`).
pandas, NumPy	Python libraries for tables and numbers	Read the simulation output and compute features and statistics.
scikit-learn 1.9.1	Python machine-learning library	Trains the decision tree and Logistic Regression, and computes accuracy, F1 and similar scores.
Matplotlib	Python plotting library	Draws every graph in the results.
Streamlit	Turns a Python script into a web page	An interactive app that runs real NS-3 simulations live in the browser, for demos.
bash scripts	Command-line scripts	`build.sh`, `run_all.sh` and others run the whole project with one command.

4.1 Why a simulator instead of real computers?

- **You can see inside the router.** The simulator reports the exact queue size at every moment. On the real internet this is impossible, and it is needed to label the training data.
- **Fair comparisons.** Every TCP version faces exactly the same traffic pattern.
- **Repeatable.** The same settings always give exactly the same result, byte for byte.
- **Cheap and fast.** A 60-second simulation finishes in about 1-2 seconds on a laptop, so hundreds of experiments are possible.

5. The Simulated Network

5.1 The dumbbell topology

All experiments use a **dumbbell** network, the standard setup for studying congestion. Several senders connect through fast links to router R1. R1 connects to router R2 over one slow link, the bottleneck. R2 connects to the receivers. All senders must squeeze through the bottleneck, so its queue is where congestion happens.

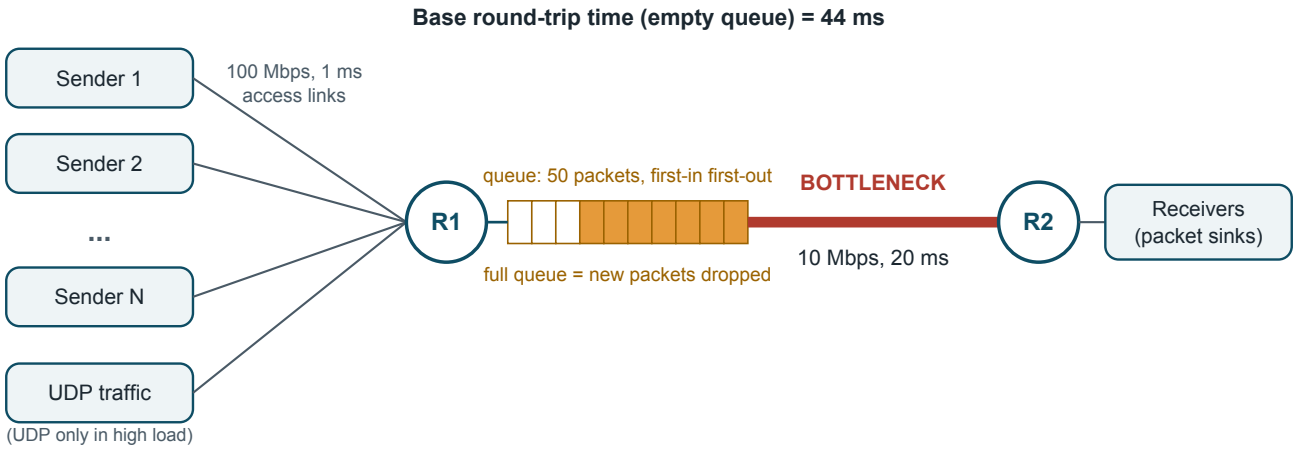


Figure 2. The simulated network (file `ns3/tcp-predictor.cc`). Orange cells represent packets waiting in R1's queue.

5.2 The three congestion levels

Level	Traffic	How busy the bottleneck is
Low	3 TCP flows that switch on and off, each at 2 Mbps	About 30%: almost no queue
Medium	6 TCP flows that switch on and off, each at 3 Mbps	About 90%, bursty: the queue fills when several flows are on together
High	8 long-running TCP flows that send as much as they can, plus 2 Mbps of on/off UDP traffic	Saturated: the queue is under constant pressure
Single	1 TCP flow alone	Used to check that a flow alone can still fill the link

A **flow** is one TCP connection from one sender to one receiver. **UDP** is a simpler protocol with no congestion control; it is added at high load as background traffic that does not slow down.

5.3 Settings shared by every TCP version

To keep the comparison fair, every version uses the same settings: 1448-byte packets, **SACK** on (the receiver reports exactly which packets arrived, which helps recovery), a minimum retransmission timeout of 1 second (the standard value from RFC 6298), 4 MB socket buffers, and **delayed ACKs** (the receiver sends one ACK for every two packets, NS-3's default). Each simulation lasts **60 seconds**.

5.4 Seeds: how the experiments are repeated

The on/off traffic uses random numbers. A **seed** is the starting point for those random numbers. The same seed always produces the same traffic, so with seed 4, Reno and Predictive TCP face exactly the same pattern of flows switching on and off. Every experiment is run with **5 seeds** (1-5) and the results are averaged.

> **Seeds are also used to keep the test honest**
>
> Seeds 1-3 are used for **everything that involves choosing**: training the model, choosing between models and tuning settings. Seeds 4-5 are kept untouched and used only to check the final result. This is like studying from practice questions (seeds 1-3) and then sitting an exam with new questions (seeds 4-5).

5.5 What is measured

Measure	What it means	Better is
Goodput	Useful data delivered per second (Mbps), not counting retransmitted copies	Higher
RTT	Average round-trip time of packets (ms); this is the delay users feel	Lower
Drops	Packets thrown away at the bottleneck because the queue was full	Lower
Retransmissions	Packets the senders had to send again	Lower
Queue	Average number of packets waiting at the bottleneck	Lower
Jain fairness index	How equally the flows share the link: (sum of rates)² ÷ (number of flows × sum of squared rates). 1.0 = perfectly equal; lower = some flows get much more than others	Closer to 1

Every 100 ms the simulation also writes one row per flow to a CSV file: the sender's smoothed RTT, lowest RTT, window, throughput, retransmissions and losses, together with the router's queue length and drops in that interval. These rows become the machine-learning dataset.

6. The Machine-Learning Part

6.1 What kind of machine learning?

This is **supervised classification**. The algorithm is shown many examples, each with inputs (called **features**) and the correct answer (called the **label**). It learns a rule that maps features to labels. Here the label has three possible values, or **classes**: **Low**, **Medium** and **High** congestion.

6.2 Building the dataset

Script: `analysis/build_dataset.py`.

1. Take the baseline simulations of **Tahoe, Reno and CUBIC** (3 congestion levels × 5 seeds). Vegas and BBR are used only for comparison, never for training.
2. Ignore the first 3 seconds of each run (start-up behaviour is unusual).
3. For every flow and every 100 ms step, compute the features from the sender's measurements.
4. Look at the **real router queue over the next 0.5 seconds** and give that moment a label:

Label	Rule (looking 0.5 s into the future)
High	The queue will reach at least 80% full (40 of 50 packets), **or** any packet will be dropped
Medium	The queue will reach at least 40% full (20 packets), but not High
Low	Neither of the above

The result is **106,354 labelled examples**. Because labels come from the future, the model learns to *predict* congestion, not just recognise it.

6.3 The features (the model's inputs)

All features come from the sender's own RTT measurements. They are divided by the base RTT so they mean the same thing on any network.

Feature	Plain meaning
`rtt_ratio`	Current RTT ÷ lowest RTT ever seen. 1.0 means the queue is empty; 1.5 means packets wait 50% longer than the travel time.
`rtt_grad`	How much the RTT changed since 100 ms ago.
`est_queue_pkts`	Estimated number of this flow's packets in the queue: window × (1 − base RTT ÷ current RTT).
`rtt_trend_05`	How much the RTT grew over the last 0.5 s.
`rtt_trend_1`	How much the RTT grew over the last 1 s.
`rtt_vs_avg`	Current RTT ÷ its own average over the last 1 s. Below 1 means the RTT is falling, so the queue is draining.

Other features were also computed and tested (loss counts, time since the last loss, the sender's window and throughput). The final model uses **only the six delay features above**, for two reasons found during the project:

- **Loss features only describe the past.** A loss means congestion *already* happened. RTT growth shows the queue filling *before* it overflows.
- **The sender's own speed creates a feedback loop.** Once the model controls the sending rate, a model that uses that rate as an input sees its own slowdown, reads it as congestion, and slows down even more. In one early test this throttled a flow to 0.6 Mbps.

6.4 What a decision tree is

A **decision tree** is a flowchart of yes/no questions, like the game "20 questions". Each question compares one feature with a threshold, such as "is rtt_ratio above 1.23?". You follow the answers down the tree until you reach a **leaf**, which gives the prediction. Training means finding the questions and thresholds that best separate Low, Medium and High in the training data.

The **depth** is the number of questions on the longest path. A deeper tree can learn more detailed rules, but it can also memorise noise in the training data instead of learning the general pattern (this is called **overfitting**).

> **Why a single decision tree and not a Random Forest?**
>
> A **Random Forest** combines hundreds of decision trees and lets them vote. It is often a little more accurate offline, but for this project it would be a poor fit:
>
> - It would have to run inside the TCP sender every 100 ms. Hundreds of trees means hundreds of times more work and far more code to export to C++.
> - It is much harder to read. A 4-leaf tree can be explained in one sentence (Section 6.5).
> - Accuracy was not the bottleneck: the model chosen was the one that worked best *inside the simulator*, which was not even the most accurate one offline (Section 6.8).
>
> So the project uses one small tree and compares it with **Logistic Regression** (a simple linear model) and a **reactive rule** ("predict High if a loss happened in the last 0.5 s").

Training settings

Script: `analysis/train_model.py`. The tree is scikit-learn's `DecisionTreeClassifier` with:

- **max_depth** chosen automatically from 2, 3, 4 and 5 by **cross-validation** on the training seeds (train on two seeds, score on the third, rotate). The smallest depth within 0.005 of the best score is kept. Depth 2 won (score 0.621).
- **min_samples_leaf = 50**: every leaf must cover at least 50 examples, which stops the tree from learning tiny, accidental patterns.
- **class_weight = "balanced"**: the classes are not equally common, so rarer classes count more during training. Otherwise the tree could just predict the most common class most of the time.
- **random_state = 0**: makes training repeatable.

6.5 The tree that was learned

After training, the tree uses only two of the six features. In plain words:

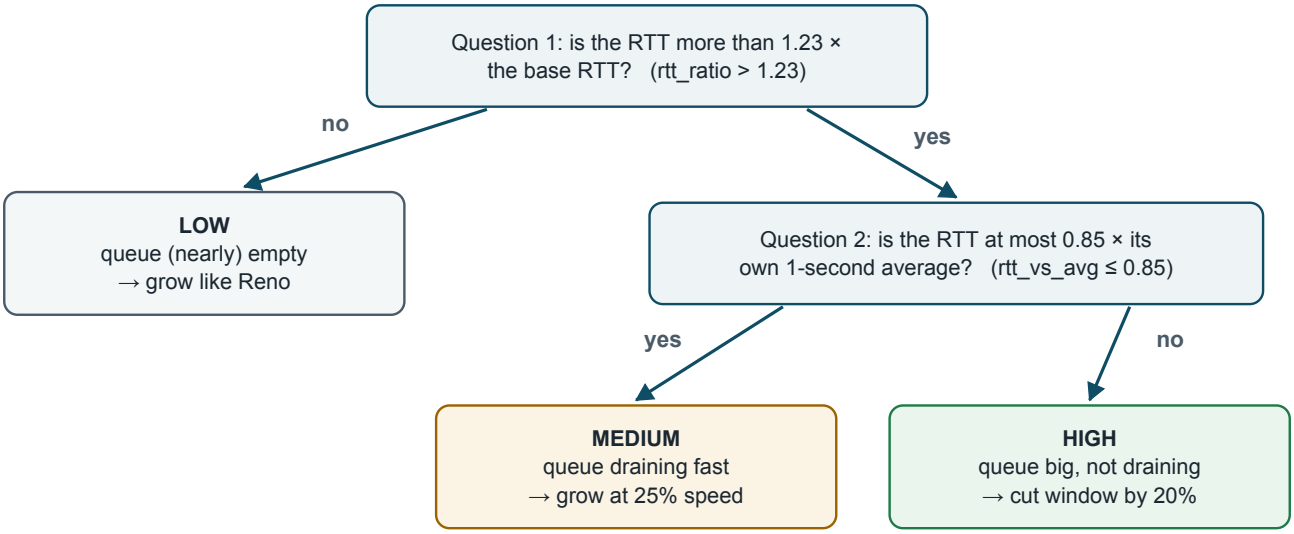


Figure 3. The deployed decision tree, simplified. (The tree as learned has one extra split inside the Low branch, but both of its leaves predict Low, so it changes nothing.)

What it learned: congestion is coming when the RTT is more than about 23% above the base RTT and is not falling quickly. With a 44 ms base RTT, 23% is about 10 ms of waiting, or roughly 8 packets in the queue. The second question separates a queue that is already draining fast (Medium) from one that is not (High).

The four leaves of the tree as scikit-learn learned it (training data):

Leaf	Path through the tree	Share of training examples	Class mix Low / Medium / High*	Prediction
1	rtt_ratio ≤ 1.16	6.1%	0.55 / 0.31 / 0.14	Low
2	1.16 < rtt_ratio ≤ 1.23	15.1%	0.91 / 0.05 / 0.05	Low
3	rtt_ratio > 1.23 and rtt_vs_avg ≤ 0.85	7.8%	0.18 / 0.64 / 0.18	Medium
4	rtt_ratio > 1.23 and rtt_vs_avg > 0.85	71.0%	0.08 / 0.39 / 0.53	High

* After class balancing, as shown in results/graphs/ml_decision_tree.png. The prediction is the class with the largest share in that leaf.

6.6 Training and test data

The examples from seeds 1-3 (**64,119**) are used for training; seeds 4-5 (**42,235**) are kept for testing. Splitting by seed, not by random rows, matters: neighbouring 100 ms rows are almost identical, so a random split would let the model see near-copies of its test questions.

6.7 How good is the model?

First, the scores used:

- **Accuracy:** the share of all predictions that are correct.
- **Precision (High):** when the model says High, how often it is right.

- **Recall (High):** of all the moments that really were High, how many the model caught.
- **F1:** one number combining precision and recall. **Macro-F1** is the average F1 over the three classes, treating Low, Medium and High as equally important.
- **Bursts warned ≥ 200 ms:** of all bursts of packet drops, the share the model warned about at least 200 ms in advance. This is what matters most for a controller.
- **False alarms:** how often the model says High when the queue stays fine.

Model (test seeds 4-5)	Accuracy	Macro-F1	Precision High	Recall High	Warned ≥ 200 ms	False alarms
Decision Tree	**0.804**	**0.605**	0.905	**0.863**	**95%**	20.5%
Logistic Regression	0.650	0.539	0.930	0.622	85%	13.5%
Reactive (loss) rule	0.547	0.364	0.915	0.498	38%	11.9%

The tree wins clearly on accuracy, recall and early warning. Its cost is more false alarms, which show up later as a small loss of speed. Its macro-F1 (0.61) is below the project's original target of 0.70, mainly because the Medium class is hard to predict: at medium load, congestion often begins when *another* flow switches on, which one sender cannot foresee.

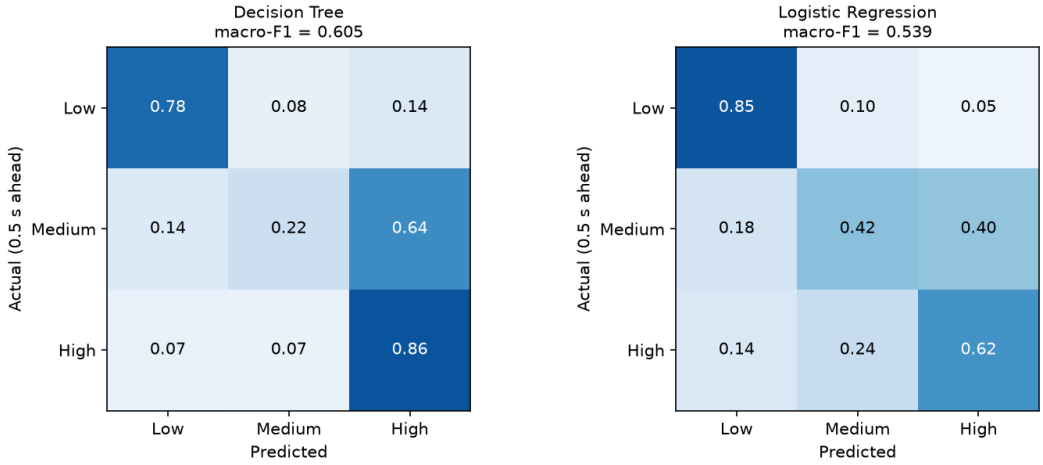


Figure 5. Confusion matrices: rows are the true class, columns the predicted class. A perfect model would have everything on the diagonal.

6.8 Choosing the model inside the simulator

In Phase 6 the team tried several **feature sets** (loss only, delay only, a mixed set, everything) and **horizons** (0.2, 0.5 and 1.0 s). Offline, the best scorer was "all features, 0.2 s" (macro-F1 0.68).

But a good score on recorded data does not guarantee good control, because once the model drives the sender it changes the traffic it is predicting. So the 5 best candidates were each **compiled into Predictive TCP and simulated** on the training seeds. The rule, written down before running: keep candidates with at least 95% of Reno's goodput and fairness of at least 0.9, then pick the largest reduction in drops plus RTT.

Candidate (in NS-3, seeds 1-3)	Drops vs Reno (med / high)	RTT vs Reno (med / high)
Phase 2 feature set, 0.5 s	−42% / −60%	−3% / 0%
All features, 0.5 s	−60% / −50%	−3% / −2%
All features, 0.2 s (best offline)	−63% / −66%	−9% / −14%
All features, 1.0 s	−46% / −55%	−3% / 0%
Delay only, 0.5 s (chosen)	**−88% / −87%**	**−24% / −32%**

The delay-only model had the *lowest* offline score yet was by far the best controller. The reason: it warns about 95% of drop bursts at least 200 ms ahead, against about 67% for the loss-based models. **Early warning matters more than getting the rare Medium class right.**

6.9 Turning the tree into C++

Script: `analysis/export_tree.py`. It reads the trained tree and writes it as C++ code into `ns3/predictive-model.h`. It then compiles a small test program and checks that the C++ version gives **exactly the same answer as scikit-learn on all 42,235 test examples**. The generated code (simplified) is:

```
// Generated by analysis/export_tree.py -- do not edit by hand.
// Returns 0 = Low, 1 = Medium, 2 = High (congestion 0.5 s ahead)
inline int PredictCongestion(double rtt_ratio, double rtt_vs_avg /* ... */)
{
    if (rtt_ratio <= 1.2333)      return 0;   // Low
    else if (rtt_vs_avg <= 0.8548) return 1;  // Medium
    else                           return 2;  // High
}
```

7. How Predictive TCP Works

Predictive TCP is a new C++ class, `TcpPredictive`, in `ns3/tcp-variants.cc`. It is built **on top of Reno**: everything Reno does still happens, and the predictor adds an early, gentle reaction on top.

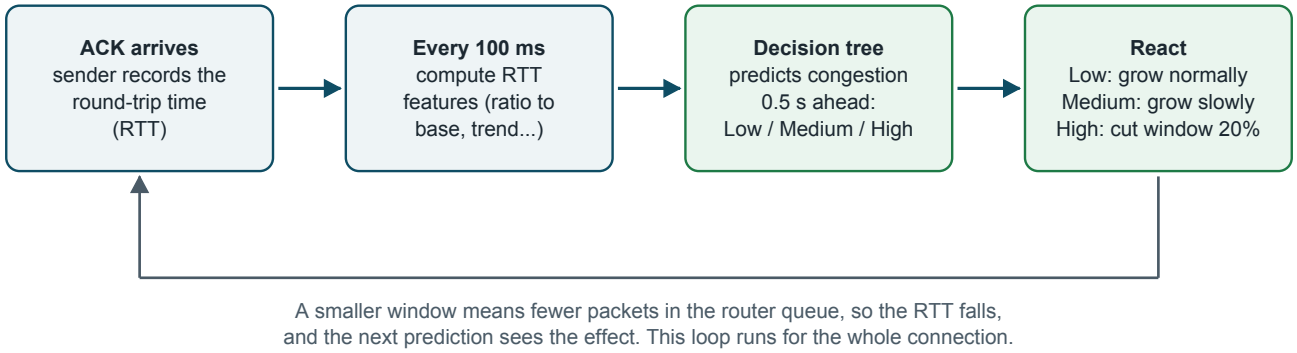


Figure 6. The control loop inside every Predictive TCP sender.

7.1 Step by step

1. **On every ACK** the sender records the RTT and keeps track of the lowest RTT it has ever seen (the base RTT).
2. **Every 100 ms** it computes the features (RTT ÷ base RTT, RTT ÷ its 1-second average, and the others) exactly the way the dataset computed them.
3. It calls `PredictCongestion()`, the C++ tree, which returns Low, Medium or High.
4. It reacts according to the table below.

Prediction	What Predictive TCP does	Why
Low	Grows the window exactly like Reno	No congestion coming; use the link fully
Medium	Grows the window at only **25%** of Reno's speed	The queue is draining; growing fast would refill it, but freezing completely was found to be unfair to other flows
High	**Cuts the window to 80%** (a 20% cut, β = 0.8) and stops growing until the prediction changes	Congestion is coming in 0.5 s; back off before the queue overflows

7.2 Safety rules

- **At most one cut per round trip.** Otherwise it could cut several times before the first cut has any effect.
- **Never cut while TCP is already recovering from a loss.** Reno is already handling it.
- **Never act below a 4-packet window (MinWindow = 4).** With a tiny window, delayed ACKs make the RTT look huge (about 240 ms at a 1-packet window), which the model would mistake for congestion. This "delayed-ACK trap" was found in Phase 3.
- **Losses are still handled by Reno** (the window is halved), as a safety net.

7.3 The competition detector

Testing showed a problem: when Predictive TCP shares the link with **CUBIC**, CUBIC keeps filling the queue until packets drop. Predictive TCP sees the queue growing, backs off, and CUBIC takes the space it freed. Predictive TCP ended up with only 23% of the bandwidth.

The fix uses a simple observation. When only Predictive TCP flows share the link, their cuts let the queue **drain now and then**, so the lowest RTT regularly drops back near the base RTT. A loss-based sender like CUBIC keeps the queue full, so the RTT never comes back down.

> **The detector rule (values tuned on seeds 1-3)**
> If even the **lowest** RTT seen over the last **2 seconds** stays above **1.4 × the base RTT** (with a 44 ms base: above about 62 ms), the queue never drained. A loss-based competitor is assumed and Predictive TCP **behaves exactly like plain Reno**. It stays in this "Reno mode" until **10 seconds** after the last such evidence.

This raised Predictive TCP's share against CUBIC from 23% to 36%. With only Predictive flows and the normal 50-packet queue, the detector never switches on, so the main results are unchanged.

7.4 Settings you can change

Setting	Default	Meaning
`Beta`	0.8	Window multiplier on a High prediction (0.8 = cut by 20%)
`MediumGrowth`	0.25	Share of Reno's growth allowed on Medium
`MinWindow`	4	Window (in packets) below which the predictor never acts
`CompetitionWindow`	2 s	Detector look-back window (0 s turns it off)
`CompetitionThreshold`	1.4	Lowest RTT ÷ base RTT above which a competitor is assumed
`CompetitionHold`	10 s	How long to stay in Reno mode after the last evidence

8. How the Project Was Built, Phase by Phase

The project was built in phases. Each phase was checked, and its results were confirmed to be exactly reproducible, before the next one started.

Phase	What was added	Outcome
1. Simulation	The dumbbell network in NS-3 with low, medium and high congestion. Tahoe and classic Reno **written from scratch** in C++ because NS-3 lacks them.	45 baseline runs; Tahoe and Reno behaviour checked with window traces
2. Dataset + ML	Logging every 100 ms, labels from the real queue 0.5 s later, Decision Tree vs Logistic Regression vs a loss rule, automatic export to C++.	106,354 samples; C++ export matches scikit-learn on all 42,235 test samples
3. Predictive TCP	`TcpPredictive`: the tree inside the sender. Two real problems found and fixed: the feedback loop and the delayed-ACK trap.	All 9 success criteria in the project's requirements document met
4. Results	Graphs, the final comparison table, the results page and the one-command pipeline `run_all.sh`.	Whole project reproducible byte for byte
5. Stronger baselines	Vegas (delay-based) and BBR (Google's model-based TCP) added.	Beats BBR on every measure; Vegas is the strongest rival
6. Stronger ML	RTT-trend features; horizons 0.2 / 0.5 / 1.0 s; model chosen by testing 5 candidates **inside the simulator**.	Drops −88%, RTT −24% to −32% vs Reno
7. Controller tuning	Cut size β and MinWindow tuned over 18 combinations on training seeds; a single-flow check added.	Drops −91% / −90%, RTT −26% / −37%
8. Robustness	Other queue sizes (20 and 100 packets) and sharing the link with CUBIC.	Gains hold for every queue size; only 23% share against CUBIC
8b. Competition detector	Switch to Reno behaviour while a loss-based sender keeps the queue full.	Share against CUBIC 23% → 36%; main results unchanged
App	Streamlit web app that runs real NS-3 simulations live.	Used for demos
9. Report + slides	Planned	Not started yet

8.1 Problems found along the way, and how they were solved

Many of the most useful lessons came from things that went wrong. Each is described in the README's "Key findings" section.

Problem	What happened	Solution
NS-3 has no Tahoe or classic Reno	Two of the five baselines did not exist	Written as small C++ recovery classes and checked with window traces
CUBIC misbehaved without SACK	One CUBIC flow reached only 4.66 Mbps with 10,315 needless retransmissions (9.48 Mbps with SACK)	SACK turned on for all versions
Tahoe froze	A 200 ms minimum timeout equals the receiver's delayed-ACK timer, so a 1-packet window timed out on every packet	Standard 1 s minimum timeout (RFC 6298)
Feedback loop	A model using the sender's own speed saw its own slowdown as congestion; one run fell to 0.6 Mbps	Model uses only network-state (RTT) features
Delayed-ACK trap	At a 1-packet window, RTT looked like ~240 ms, a false congestion signal	Predictor never acts below a 4-packet window
Freezing on Medium was unfair	Fewest drops, but fairness fell to 0.78	Grow at 25% speed on Medium (fairness back to 0.99)
Best offline model ≠ best controller	The top-scoring model gave much smaller gains in the simulator	Choose models by testing inside NS-3
Halving looked best but hurt a lone flow	β = 0.5 left a single flow at 87% of Reno's speed	Single-flow check added to the rule; β = 0.8 chosen
Losing to CUBIC	Only 23% of the bandwidth	Competition detector (36%)

8.2 Rules followed throughout

- **No peeking at the test data.** Every choice (model, settings, detector) used seeds 1-3 only.
- **Selection rules written down before running.** When a rule was changed after seeing a result (the single-flow check in Phase 7), this is stated openly in the code and the README.
- **No hiding weaknesses.** Every weakness found is reported (Section 10).
- **Reproducibility checked after every phase** by comparing file checksums.

9. Results

All numbers are the average of 5 seeds of 60-second simulations (10 Mbps / 20 ms bottleneck, 50-packet queue), as reported in `results/RESULTS.md`.

9.1 Headline: Predictive TCP compared with Reno

Compared with Reno	Medium congestion	High congestion
Packets dropped at the bottleneck	**−91%**	**−90%**
Retransmissions	**−91%**	**−89%**
Average RTT (delay)	**−26%** (59 vs 80 ms)	**−37%** (64 vs 101 ms)
Goodput (speed)	98.0% of Reno	99.6% of Reno
Fairness (Jain index)	0.98 (Reno 0.98)	0.96 (Reno 0.99)

- **Every seed:** Predictive TCP has fewer drops and lower RTT than Reno, CUBIC and Tahoe in all 5 seeds at both medium and high load.
- **Alone on the link** it reaches 9.7 Mbps of 10 (Reno 9.0).
- **At low load** all six versions behave the same, because there is no congestion to avoid.
- **The costs:** about 2% less goodput at medium load, and slightly lower fairness at high load (0.96 vs 0.99).

9.2 All six versions side by side

Load	Version	Goodput (Mbps)	RTT (ms)	Drops	Queue (pkts)	Fairness
Medium	Tahoe	8.30	80.4	571	23.8	0.981
Medium	Reno	8.32	79.9	701	22.4	0.982
Medium	CUBIC	8.32	84.2	632	24.3	0.981
Medium	Vegas	8.13	65.0	**9**	6.5	0.984
Medium	BBR	8.22	87.5	2398	26.3	0.980
Medium	**Predictive**	8.16	**59.2**	61	**6.4**	0.982
High	Tahoe	8.54	96.5	1568	39.8	0.991
High	Reno	8.65	101.4	1623	43.2	0.994
High	CUBIC	8.70	109.0	1376	46.9	0.987
High	Vegas	8.66	106.4	277	41.9	0.882
High	BBR	8.58	108.2	7886	47.7	0.854
High	**Predictive**	8.62	**64.4**	**163**	**14.8**	0.955

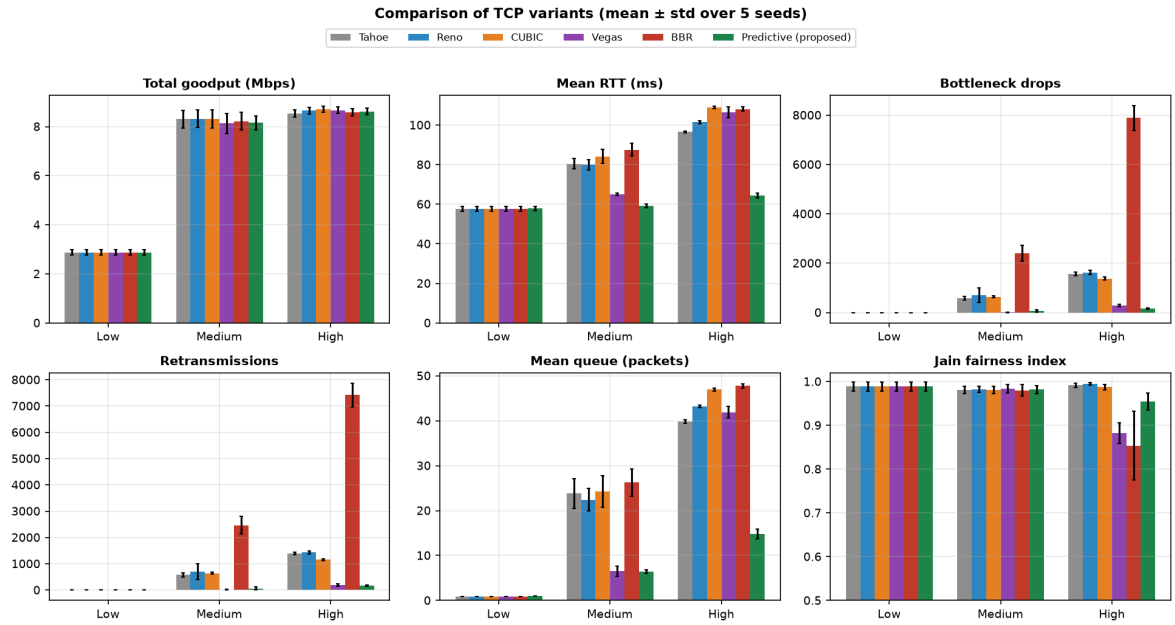


Figure 7. All six versions compared (green = Predictive TCP). Error bars show the spread over the 5 seeds.

9.3 Against the delay-based and model-based versions

- **vs BBR:** drops −97% / −98% and RTT −32% / −40% (medium / high). BBR does not slow down on loss, so 8 BBR flows overflow the small 50-packet queue (7,886 drops at high load, 5 × Reno). This matches published measurements of BBR v1.
- **vs Vegas**, the strongest rival: at medium load Vegas loses fewer packets (9 vs 61), but Predictive TCP has lower RTT (59 vs 65 ms). At high load Predictive TCP is better in every seed: drops −41%, RTT −40%, fairness 0.96 vs 0.88.
- **Vegas alone cannot fill the link:** one Vegas flow reaches only 5.3 Mbps of 10, because delayed ACKs add a few ms to every RTT sample and Vegas reads that as queueing. Predictive TCP reaches 9.7.

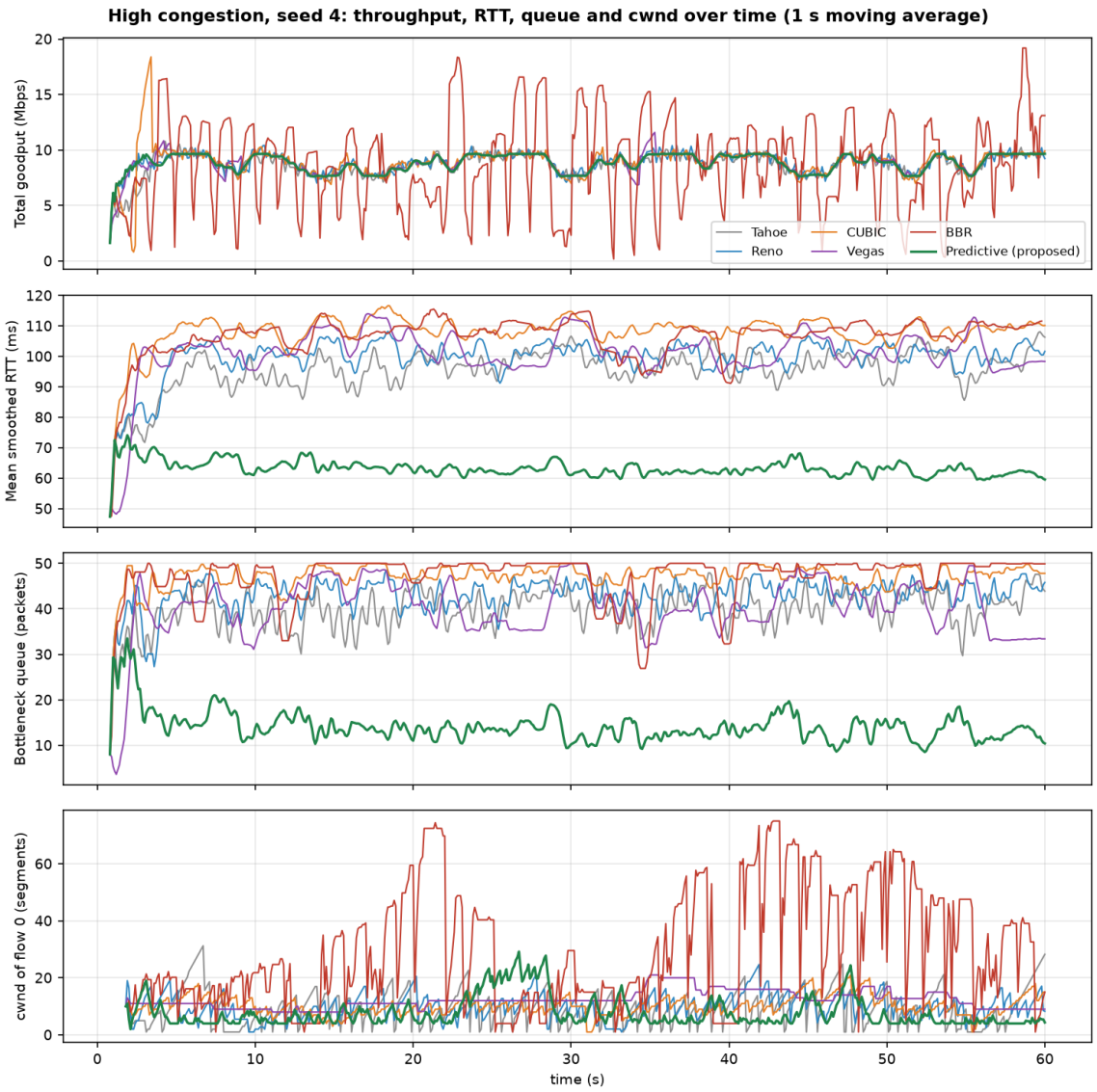


Figure 8. One high-load run over 60 seconds (seed 4, never used for training). The thick green line is Predictive TCP. Its speed (top) matches the others, but its RTT (second panel) and queue (third panel) stay far lower, because it backs off before the queue fills. BBR (red) swings wildly and keeps the queue full.

9.4 Is it overfitted? The unseen seeds

Everything was chosen on seeds 1-3. If the design only worked on those, the gains would shrink on seeds 4-5. They do not:

Load	Seeds	Drops vs Reno	RTT vs Reno	Goodput vs Reno	Fairness
Medium	1-3 (used for choices)	−90.7%	−25.8%	98.1%	0.985
Medium	4-5 (never seen)	−92.4%	−26.1%	97.8%	0.977
High	1-3 (used for choices)	−89.8%	−36.0%	99.7%	0.960
High	4-5 (never seen)	−90.2%	−37.2%	99.5%	0.947

9.5 Success criteria

The project's requirements document (`docs/PRD.md`) set targets before the work began. **All 9 targets for Predictive TCP are met**: at least 30% fewer drops than Reno, lower RTT than Reno and CUBIC, at least 90% of Reno's goodput (each at medium and high load), and fairness of at least 0.9 at high load. For the prediction model, **3 of 4 targets are met**; macro-F1 is 0.61 against a target of 0.70.

9.6 Which part matters? (ablation)

An **ablation** removes one part at a time to see what it contributes. Removing the 20% proactive cut (β = 1.0) at high load raises drops from 163 to 267, RTT from 64 to 106 ms, and drops fairness to 0.82. **The proactive cut is the main mechanism.** Turning off the special Medium reaction changes almost nothing, because the tree rarely predicts Medium.

9.7 Different queue sizes

The model was trained and tuned with a 50-packet queue only. With 20 or 100 packets:

Queue	Load	Drops vs Reno	RTT vs Reno	Goodput vs Reno
20	Medium / High	−86% / −88%	−10% / −17%	98% / 100%
50	Medium / High	−91% / −90%	−26% / −37%	98% / 100%
100	Medium / High	−96% / −62%	−43% / −24%	98% / 100%

The bigger the queue, the bigger the delay gain, because loss-based TCP fills whatever queue exists ("bufferbloat") while Predictive TCP keeps it short. With 100 packets at high load the gain is smaller than expected because the competition detector false-alarms there (Section 10); with the detector off the result would be −87% drops and −50% RTT.

9.8 Sharing the link with CUBIC

At high load, 4 flows of one version share the link with 4 CUBIC flows. A fair version gets 50%.

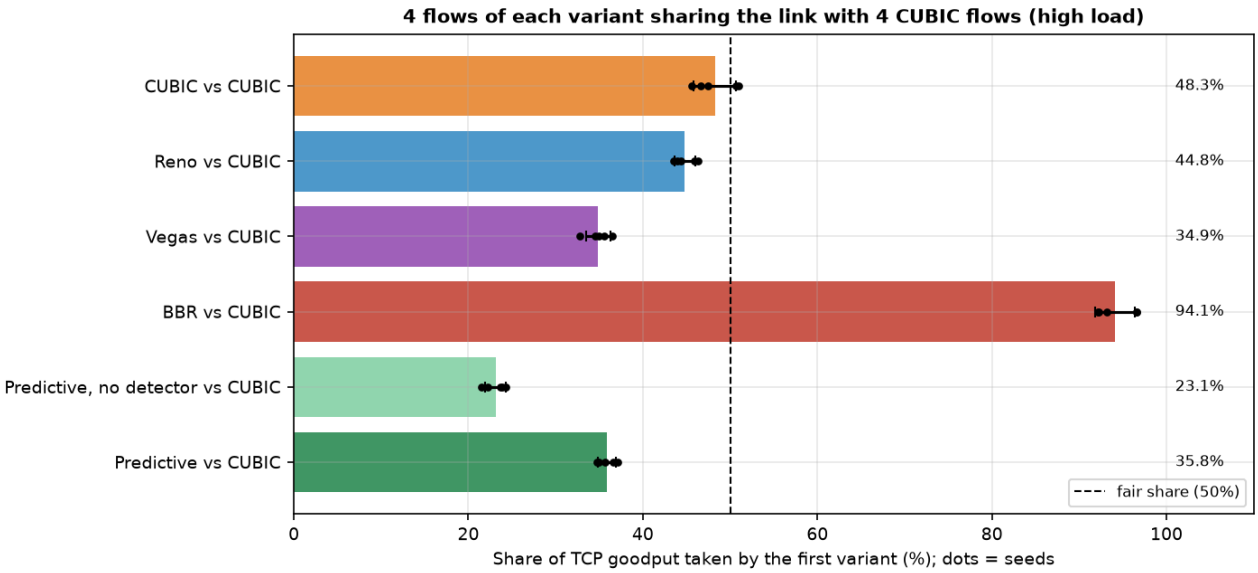


Figure 9. Share of the bandwidth taken by each version against CUBIC. Without the detector Predictive TCP gets 23%; with it, 36%, about the same as Vegas (35%). BBR takes 94%, the opposite problem.

This is the classic weakness of delay-based congestion control: a sender that backs off when the queue grows loses out to one that does not. Settings that reach 40-44% exist, but they also switch to Reno mode when no competitor is present, which destroys most of the gains, so the tuning rule rejected them.

10. Honest Limitations

- **Simulation only.** One bottleneck and one FIFO queue. Real networks are more varied, and the results may differ there.
- **Offline macro-F1 is 0.61, below the 0.70 target.** The Medium class is predicted poorly (F1 0.20). Recall for High (0.86) and early warning do meet their targets.
- **The learned rule is close to a delay-based controller.** Like Vegas, it acts on queueing delay. The differences are that its threshold and "is the queue draining?" test are learned from data, and its reaction is a 20% cut on top of Reno instead of Vegas's ±1 packet adjustments.
- **Fairness at high load is slightly lower than Reno's** (0.96 vs 0.99), though above the 0.9 target.
- **Tuned for one topology.** It held up for 20- and 100-packet queues, but link speed, RTT and the number of flows were not varied. β and MinWindow may need other values elsewhere.
- **Vegas still has fewer drops at medium load.**
- **Not fully competitive against loss-based TCP:** 36% of the bandwidth against CUBIC, below Reno's 45%. It suits networks where most senders behave alike (a data centre or a managed network) better than the open internet.
- **The competition detector can false-alarm.** With a 100-packet queue at high load it switched to Reno mode for 60-83% of the time in 3 of 5 seeds, though no CUBIC flow was present. The alarm feeds itself: one flow in Reno mode keeps the queue full, so the others think a competitor is there too. Predictive TCP still beat Reno in every seed, but by less. The detector can be switched off (`CompetitionWindow=0s`). A possible fix: briefly drain the queue now and then while in Reno mode to check whether a competitor is really present, as BBR's ProbeRTT does.

11. How to Run It Yourself

11.1 What is in the project folder

Path	What it contains
`ns3/tcp-predictor.cc`	The simulated network, the congestion levels and the 100 ms logging
`ns3/tcp-variants.h/.cc`	Tahoe, Reno and Predictive TCP (C++)
`ns3/predictive-model.h`	The decision tree as C++ (generated, do not edit)
`analysis/build_dataset.py`	Features + labels → results/dataset.csv
`analysis/ml_experiments.py`	Feature sets × horizons (offline)
`analysis/select_model.py`	Tests candidate models inside NS-3; picks one
`analysis/train_model.py`	Trains the tree; compares with Logistic Regression and the loss rule
`analysis/export_tree.py`	Tree → C++, verified against scikit-learn
`analysis/tune_controller.py`	Tunes β and MinWindow inside NS-3
`analysis/tune_competition.py`	Tunes the competition detector
`analysis/compare_variants.py`	Metrics, success-criteria check, ablation
`analysis/robustness.py`	Queue-size sweep and CUBIC competition
`analysis/make_graphs.py`	All graphs, final table and results/RESULTS.md
`results/`	All output: tables, graphs, trained models, raw runs
`reference/`	The exact macOS simulation data, for identical results on any computer
`app/`	The interactive Streamlit simulator
`docs/`	Project plan and requirements document (PRD)
`build.sh, run_all.sh`	Compile the simulation; run everything

11.2 One-time setup (macOS)

```
xcode-select --install            # C++ compiler and git
brew install cmake ninja uv       # build tools (Homebrew: https://brew.sh)

# NS-3.48 into ~/ns-3 (about 10-15 minutes, once)
mkdir -p ~/ns-3 && cd ~/ns-3
curl -L https://gitlab.com/nsnam/ns-3-dev/-/archive/ns-3.48/\
ns-3-dev-ns-3.48.tar.gz | tar xz --strip-components=1
./ns3 configure -d optimized --disable-examples --disable-tests \
  --enable-modules="core;network;internet;point-to-point;\
applications;traffic-control;flow-monitor"
./ns3 build
cd -

# Python environment for the analysis (from the project folder)
uv venv .venv --python 3.13
uv pip install --python .venv/bin/python -r requirements.txt
```

Linux (Ubuntu/Debian) and Windows (through WSL2) follow the same steps with `apt` instead of Homebrew; see the README's Setup section.

11.3 Run everything

```
./build.sh       # copy ns3/ into NS-3 and compile
./run_all.sh     # reproduce every result (about 16-18 minutes)
```

Step	What run_all.sh does	Time
1	Compile the simulation	~20 s
2	75 baseline runs (5 versions × 3 loads × 5 seeds) + validation traces	~1.5 min
3	Build the dataset (106,354 samples)	~10 s
4	Feature/horizon experiments, then pick the model inside NS-3	~1.5 min
5	Train and evaluate the tree vs Logistic Regression vs the loss rule	~10 s
6	Export the tree to C++, verify, recompile	~30 s
7	Tune β and MinWindow inside NS-3	~3 min
8	Tune the competition detector inside NS-3	~7 min
9	Predictive TCP runs + ablation	~1 min
10	Robustness: queue sizes and competition with CUBIC (155 runs)	~3.5 min
11	Tables, graphs and results/RESULTS.md	~30 s
12	Compare every result with the macOS reference	~5 s

Every step is deterministic: running it twice gives byte-identical files on the same machine.

11.4 Identical results on any computer (reference mode)

Tiny floating-point differences between operating systems and processors change which packet a router drops, so a re-simulation on Linux gives very close but not identical numbers. To get exactly the macOS results anywhere, the repository includes the original simulation output (`reference/simulation_data.tar.xz`, 15 MB):

```
./run_all.sh --reference     # about 1 minute, no NS-3 needed
# ends with: RESULT: every result is IDENTICAL to the macOS reference.
```

11.5 Run a single simulation

```
cd ~/ns-3
./ns3 run "tcp-predictor --variant=predictive --level=high --seed=1 --outDir=/tmp"
./ns3 run "tcp-predictor --variant=cubic --level=medium --seed=4 --outDir=/tmp"

# share the link with CUBIC
./ns3 run "tcp-predictor --variant=predictive --mix=cubic --level=high --outDir=/tmp"
# a bigger router queue
./ns3 run "tcp-predictor --variant=predictive --level=high --queue=100 --outDir=/tmp"
```

Options: `--variant` (tahoe, reno, cubic, vegas, bbr, predictive), `--level` (low, medium, high, single), `--seed`, `--simTime`, `--rate`, `--delay`, `--queue`, `--mix`, and the Predictive TCP settings from Section 7.4 (for example `--ns3::TcpPredictive::Beta=0.7`). Each run prints a one-line summary and writes three CSV files.

11.6 The interactive app

```
uv venv app/.venv --python 3.12
uv pip install --python app/.venv/bin/python -r app/requirements.txt
./app/run_app.sh      # opens http://localhost:8501 ; stop with Ctrl + C
```

The app has three tabs. **Live simulation** runs real NS-3 simulations (about 1-2 s each) with your choice of versions, load, seed, link and Predictive TCP settings, and shows a results table and four charts over time (goodput, RTT, queue, window). **Project results** shows the final table and graphs. **How it works** shows the tree, the detector and the generated C++ code.

A good demo in four steps

1. Compare versions, High load, seed 4: Predictive TCP has about 90% fewer drops and 37% lower RTT than Reno; the queue chart shows why.
2. Add Vegas and BBR: BBR floods the queue; Vegas loses fewer packets but has higher RTT.
3. Set the queue size to 100: the RTT gap grows (bufferbloat).
4. Share with CUBIC, Predictive, detector on: about 35%. Turn the detector off: about 22%.

12. Quick Questions and Answers

Which machine-learning model does the project use?

A single Decision Tree (scikit-learn DecisionTreeClassifier), depth 2, 4 leaves. Not a Random Forest. It was compared with Logistic Regression and a simple loss-based rule.

Why not a Random Forest or a neural network?

The model runs inside every TCP sender every 100 ms and has to be exported to C++. A 4-leaf tree is a few if/else lines, essentially free to run, and fully explainable. The bigger models would cost more and be much harder to explain, and the deciding test was performance inside the simulator, not offline accuracy.

What does the model actually predict?

Whether the bottleneck queue will be Low, Medium or High (at least 80% full, or dropping packets) during the next 0.5 seconds.

Does the sender need information from the router?

No. The model uses only the sender's own RTT measurements. The router's queue was used only to label the training data.

What are its inputs, in one sentence?

How much higher the current RTT is than the base RTT (rtt_ratio), and whether the RTT is falling compared with its recent average (rtt_vs_avg).

What does Predictive TCP do on a High prediction?

It cuts its congestion window to 80% (β = 0.8), at most once per round trip, never below 4 packets, and stops growing until the prediction clears.

How do you know it is not overfitted?

All choices were made on seeds 1-3. On the untouched seeds 4-5, the gains are the same size (drops −92% / −90%, RTT −26% / −37%).

Why 0.5 seconds?

0.2, 0.5 and 1.0 s were tried. Tested inside the simulator, the 0.5 s delay-only model gave by far the biggest reduction in drops and RTT.

Why a minimum window of 4 packets?

At very small windows delayed ACKs inflate the RTT (to about 240 ms at 1 packet), which looks like congestion. Below 4 packets the predictor stays quiet.

Is Predictive TCP just TCP Vegas?

It is related, since both react to queueing delay. But the threshold and the draining test are learned from data, and the reaction is a 20% cut on top of Reno instead of Vegas's ±1 packet changes. It beats Vegas at high load, and fills a link alone (9.7 vs 5.3 Mbps).

What is its biggest weakness?

Sharing a link with CUBIC: 36% of the bandwidth with the competition detector (23% without), where 50% would be fair.

Can the results be reproduced?

Yes. ./run_all.sh regenerates everything byte for byte on the same machine, and ./run_all.sh --reference gives the exact macOS results on any computer in about a minute.

13. Glossary

ACK	Acknowledgement: a small message from the receiver confirming which data arrived.
Ablation	An experiment that removes one part of a design to measure what that part contributes.
Bandwidth	How many bits per second a link can carry (here 10 Mbps at the bottleneck).
BDP	Bandwidth-delay product: packets that fit in flight during one round trip (here ≈ 38).
Base RTT	The RTT with an empty queue; the lowest RTT the sender has seen (here 44 ms).
Bottleneck	The slowest link on the path; where the queue builds up.
Bufferbloat	Very large queues that fill up and add a lot of delay.
Class	One possible answer of a classifier (here Low, Medium or High).
Congestion	More traffic arriving at a link than it can carry, so a queue builds up.
Congestion window (cwnd)	How many packets a TCP sender may have unacknowledged at once; controls its speed.
Cross-validation	Scoring a model by training on part of the data and testing on the rest, rotating which part is held out.
Decision tree	A model that predicts by asking a series of yes/no questions about the features.
Delayed ACK	The receiver waits briefly and acknowledges two packets with one ACK.
Drop	A packet thrown away because the queue was full.
Dumbbell topology	Many senders and receivers connected by one shared bottleneck link.
Feature	An input measurement given to the model.
FIFO	First in, first out: packets leave the queue in the order they arrived.
Flow	One TCP connection from a sender to a receiver.
Goodput	Useful data delivered per second, excluding retransmitted copies.
Jain fairness index	A number from 0 to 1 showing how equally flows share a link; 1 is perfectly equal.
Label	The correct answer attached to a training example.
Logistic Regression	A simple model that predicts classes from a weighted sum of the features.
Macro-F1	The average F1 score over all classes, each class counting equally.
NS-3	A free discrete-event network simulator used in research and teaching.
Overfitting	When a model learns noise in its training data and does worse on new data.
Precision / recall	Precision: how often a High prediction is right. Recall: how many real High moments were caught.
Random Forest	Many decision trees that vote together. Not used in this project.
Retransmission	Sending a lost packet again.
RTO	Retransmission timeout: how long the sender waits before assuming a packet is lost.
RTT	Round-trip time: time from sending a packet to receiving its ACK.
SACK	Selective acknowledgement: the receiver says exactly which packets arrived.

Seed	The starting point for the simulator's random numbers; the same seed gives the same traffic.
Slow start	TCP's start-up phase, where the window doubles every round trip.
UDP	A simple protocol with no congestion control, used here as background traffic.
β (beta)	In Predictive TCP, the window multiplier on a High prediction (0.8).

References

V. Jacobson, "Congestion Avoidance and Control," ACM SIGCOMM, 1988. · L. Brakmo and L. Peterson, "TCP Vegas," IEEE JSAC, 1995. · N. Cardwell et al., "BBR: Congestion-Based Congestion Control," ACM Queue, 2016. · M. Hock, R. Bless and M. Zitterbart, "Experimental Evaluation of BBR Congestion Control," IEEE ICNP, 2017. · V. Arun and H. Balakrishnan, "Copa," USENIX NSDI, 2018. · K. Tan et al., "Compound TCP," IEEE INFOCOM, 2006. · R. Jain, D. Chiu and W. Hawe, "A Quantitative Measure of Fairness," DEC-TR-301, 1984. · RFC 5681, RFC 6298, RFC 9438. · NS-3, https://www.nsnam.org

Every number in this guide comes from the project's own results (results/RESULTS.md, README.md and the files in results/model/).